



Domain Driven Design - Java

Prof. Gilberto Alexandre das Neves
profgilberto.neves@fiap.com.br

A classe ArrayList

A classe ArrayList

A classe *ArrayList* nos permite trabalhar com arrays redimensionáveis (essa classe deve ser importada do pacote `java.util`).

Os elementos de um *ArrayList* são considerados objetos. Para criar vetores dos tipos primitivos, devemos utilizar **suas classes** ao invés do tipo primitivo em si.

Fique atento que sua declaração e instanciação é levemente diferente.

Exemplos:

```
ArrayList<String> cars = new ArrayList<String>();
```

```
ArrayList<Integer> numeros = new ArrayList<Integer>();
```

```
ArrayList<Float> salarios = new ArrayList<Float>();
```

A classe ArrayList

Para inserir valores em um *ArrayList* utilizamos o método **.add()**. Exemplo:

```
cars.add("Volvo");  
cars.add("BMW");  
cars.add("Ford");
```

Para obter um valor de um *ArrayList* utilizamos o método **.get()**. Exemplo:

```
System.out.println(cars.get(1)); //exibe BMW
```

Para atribuir um novo valor para um posição já preenchida de um *ArrayList* utilizamos o método **.set(*índice*, *novo valor*)**. Exemplo:

```
cars.set(1, "Fusca"); //"troca" BMW por Fusca
```

Para remover um valor de um *ArrayList* utilizamos o método **.remove()**. Exemplo:

```
cars.remove(1); //remove Fusca do array
```

A classe ArrayList

Para obter o tamanho de um *ArrayList* utilizamos o método **.size()**. Exemplo:

```
ArrayList<String> cars = new ArrayList<String>();  
cars.add("Volvo");  
cars.add("BMW");  
cars.add("Ford");  
System.out.println(cars.size()); //exibe 3
```

Para remover todos os valores de um *ArrayList* utilizamos o método **.clear()**. Exemplo:

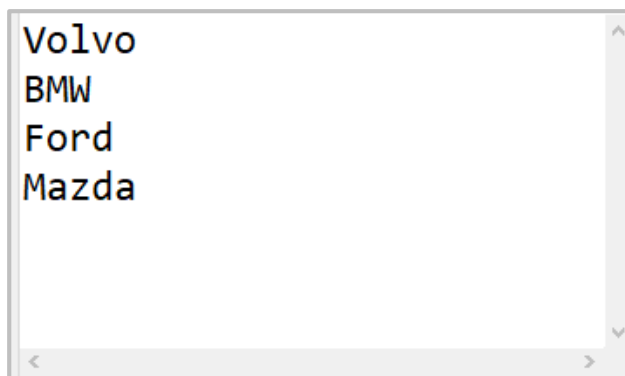
```
cars.clear();
```

Exibindo todos os valores de um ArrayList

Podemos exibir cada elemento de um *ArrayList* utilizando, por exemplo, o comando **for**. Exemplo:

```
ArrayList<String> cars = new ArrayList<String>();  
cars.add("Volvo");  
cars.add("BMW");  
cars.add("Ford");  
cars.add("Mazda");  
for (int i = 0; i < cars.size(); i++) {  
    System.out.println(cars.get(i));  
}
```

Resultado:



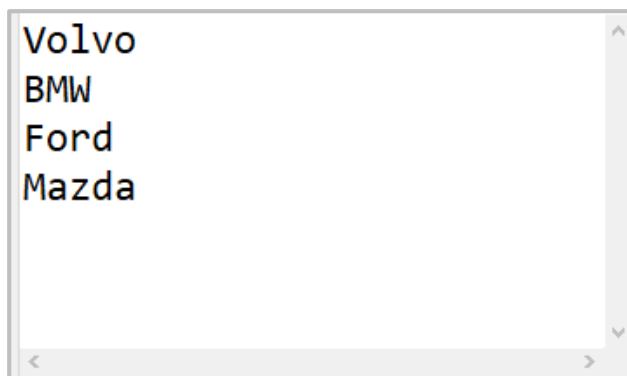
```
Volvo  
BMW  
Ford  
Mazda
```

Exibindo todos os valores de um ArrayList

Podemos exibir cada elemento de um *ArrayList* utilizando, por exemplo, o comando **for-each**. Exemplo:

```
ArrayList<String> cars = new ArrayList<String>();  
cars.add("Volvo");  
cars.add("BMW");  
cars.add("Ford");  
cars.add("Mazda");  
for (String i : cars) {  
    System.out.println(i);  
}
```

Resultado:



```
Volvo  
BMW  
Ford  
Mazda
```

A classe Collections

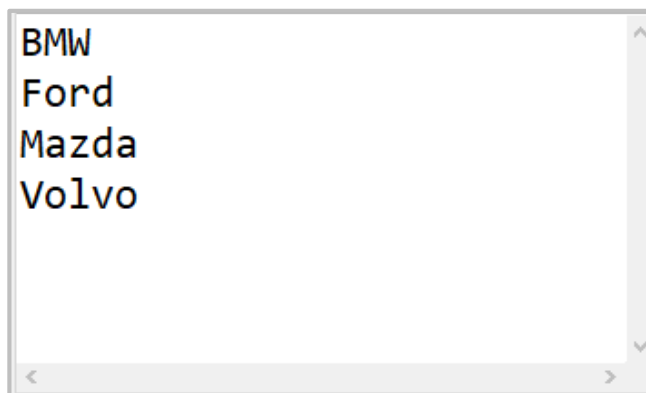
Organizando um ArrayList

Utilizando a classe *Collections* podemos utilizar o método **.sort()** que nos permite classificar os valores de um *ArrayList* (alfabeticamente ou numericamente).

Exemplo:

```
ArrayList<String> cars = new ArrayList<String>();  
cars.add("Volvo");  
cars.add("BMW");  
cars.add("Ford");  
cars.add("Mazda");  
Collections.sort(cars); //classifica alfabeticamente  
for (String i : cars) {  
    System.out.println(i);  
}
```

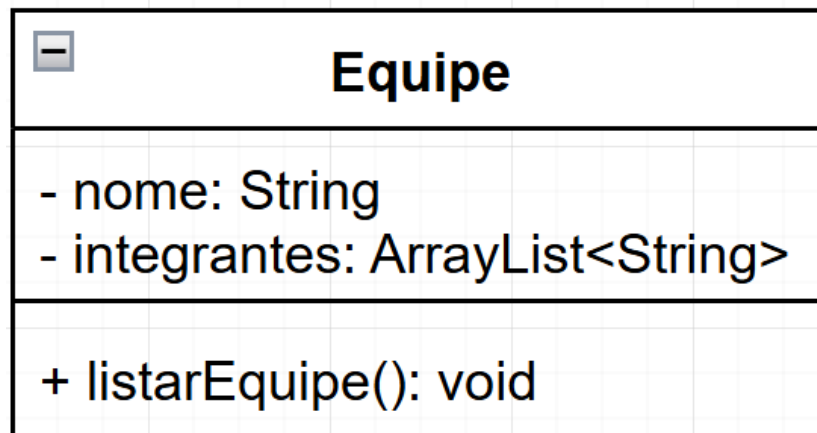
Resultado:



```
BMW  
Ford  
Mazda  
Volvo
```

A classe Equipe

Implemente a classe **Equipe** como indicado pelo modelo UML abaixo:



Orientações:

- definir um construtor vazio e outro com passagem de parâmetros.
- Definir todos os *getters* e *setters*.
- método `listarEquipe()` deve exibir o nome da equipe e todos os seus integrantes em ordem alfabética (use a classe **JOptionPane**).

A classe Equipe

```
8 public class Equipe {
9     private String nome;
10    private ArrayList<String> integrantes;
11    public Equipe() {}
12    public Equipe(String nome, ArrayList<String> integrantes) {
13        this.nome = nome;
14        this.integrantes = integrantes;
15    }
16    > public String getNome() { return nome; }
19    > public void setNome(String nome) { this.nome = nome; }
22    > public ArrayList<String> getIntegrantes() { return integrantes; }
25    > public void setIntegrantes(ArrayList<String> integrantes) { this.integrantes = integrantes; }
28    public void listarEquipe() {
29        String exibe = String.format("Nome da equipe: %s \n", nome);
30        Collections.sort(integrantes);
31        int cont = 1;
32        for(String i : integrantes) {
33            exibe += String.format("Integrante %d: %s \n", cont, i);
34            cont++;
35        }
36        JOptionPane.showMessageDialog(null, exibe, "Listagem da Equipe", JOptionPane.INFORMATION_MESSAGE);
37    }
38 }
```

A classe MainEquipe

Implemente a classe **MainEquipe** com o método **main** para podermos criar objetos da classe criada anteriormente e testar seus atributos e métodos. Veja a seguir as orientações:

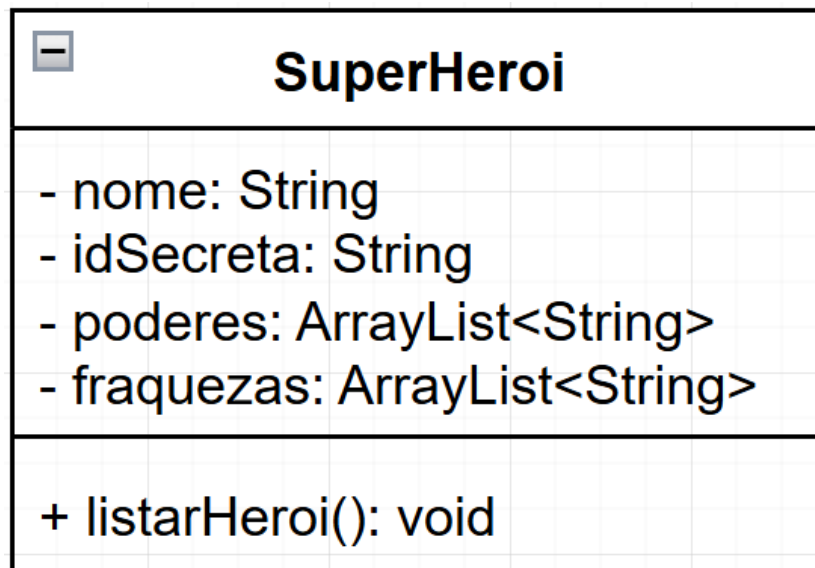
1. Peça para o usuário digitar o nome da equipe.
2. Peça para o usuário digitar o nome de cada um dos integrantes e quando quiser parar ele deve digitar “fim”.
3. Declare e instancie um objeto da classe Equipe com as informações passadas pelo usuário anteriormente.
4. Exiba a lista da equipe.
5. Perguntar ao usuário se deseja continuar (em caso afirmativo repetir os passos de 1 a 5 novamente).
6. Em caso negativo, exibir uma mensagem se despedindo do usuário e encerre o programa.

A classe MainEquipe

```
11 ▶ public static void main(String[] args) {
12     Equipe grupo;
13     String auxiliar, nome;
14     do {
15         try {
16             nome = JOptionPane.showInputDialog("Digite o nome da equipe");
17             String membros = "continua";
18             ArrayList<String> integrantes = new ArrayList<String>();
19             while (membros.equalsIgnoreCase("continua")) {
20                 auxiliar = JOptionPane.showInputDialog("Digite integrante da equipe ou \"fim\" para encerrar.");
21                 if (auxiliar.equalsIgnoreCase("fim")) {
22                     membros = "fim";
23                 } else {
24                     integrantes.add(auxiliar);
25                 }
26             }
27             grupo = new Equipe(nome, integrantes);
28             grupo.listarEquipe();
29         } catch (Exception e) {
30             JOptionPane.showMessageDialog(null, e.getMessage(), "Erro", JOptionPane.ERROR_MESSAGE);
31         }
32     } while (JOptionPane.showConfirmDialog(null, "Deseja continuar?", "Atenção", JOptionPane.YES_NO_OPTION,
33         JOptionPane.QUESTION_MESSAGE) == 0);
34     JOptionPane.showMessageDialog(null, "Fim de programa. Até breve!", "Adeus", JOptionPane.WARNING_MESSAGE);
35 }
36 }
```

Praticando...

Implemente a classe **SuperHeroi** como indicado pelo modelo UML abaixo:



Orientações:

- definir um construtor vazio e outro com passagem de parâmetros.
- Definir todos os *getters* e *setters*.
- método `listarHeroi()` deve exibir o nome, a identidade secreta, todos os poderes e todas as fraquezas (ambas em ordem alfabética) do super-herói (use a classe **JOptionPane**).

Implemente a classe **MainSuperHeroi** com o método **main** para podermos criar objetos da classe criada anteriormente e testar seus atributos e métodos. Veja a seguir as orientações:

1. Peça para o usuário digitar o nome e a identidade secreta do super-herói.
2. Peça para o usuário digitar cada poder e cada fraqueza deste super-herói e quando quiser parar ele deve digitar “fim”.
3. Declare e instancie um objeto da classe SuperHeroi com as informações passada pelo usuário anteriormente.
4. Exiba todas as informações deste super-herói.
5. Perguntar ao usuário se deseja continuar (em caso afirmativo repetir os passos de 1 a 6 novamente).
6. Em caso negativo, exibir uma mensagem se despedindo do usuário e encerre o programa.

Questionário

1. Observe o código abaixo e indique qual valor será exibido após executar essa classe?

```
ArrayList<Integer> numeros =  
    new ArrayList<Integer>();  
numeros.add(42);  
numeros.add(13);  
numeros.add(21);  
numeros.add(33);  
numeros.remove(1);  
for (Integer i : numeros) {  
    System.out.println(i + " ");  
}
```

- a) 13 21 33
- b) 42 21 33
- c) 42 13 33
- d) 42 13 21

2. Observe o código abaixo e indique qual valor será exibido após executar essa classe?

```
ArrayList<Integer> numeros =  
    new ArrayList<Integer>();  
numeros.add(42);  
numeros.add(13);  
numeros.add(21);  
numeros.add(33);  
Collections.sort(numeros);  
for (Integer i : numeros) {  
    System.out.println(i + " ");  
}
```

- a) 42 13 21 33
- b) 13 21 33
- c) 21 42 33
- d) 13 21 33 42



Java como programar. Paul Deitel e Harvey Deitel. Pearson, 2011.

Java 8 – Ensino Didático : Desenvolvimento e Implementação de Aplicações. Sérgio Furgeri. Editora Érica, 2015.

Até breve!

Questionário

1. Observe o código abaixo e indique qual valor será exibido após executar essa classe?

- a) 13 21 33
- ☒ b) 42 21 33
- c) 42 13 33
- d) 42 13 21

```
ArrayList<Integer> numeros =  
    new ArrayList<Integer>();  
numeros.add(42);  
numeros.add(13);  
numeros.add(21);  
numeros.add(33);  
numeros.remove(1);  
for (Integer i : numeros) {  
    System.out.println(i + " ");  
}
```

2. Observe o código abaixo e indique qual valor será exibido após executar essa classe?

- a) 42 13 21 33
- b) 13 21 33
- c) 21 42 33
- ☒ d) 13 21 33 42

```
ArrayList<Integer> numeros =  
    new ArrayList<Integer>();  
numeros.add(42);  
numeros.add(13);  
numeros.add(21);  
numeros.add(33);  
Collections.sort(numeros);  
for (Integer i : numeros) {  
    System.out.println(i + " ");  
}
```